

Kütüphane Yönetim Sistemi

Giriş

Basit bir kütüphane yönetim sistemi için **fonksiyonlar** (*metotlar*) tasarlamamız beklenmektedir. Aşağıda, yazmanız gereken fonksiyonların **amaçları**, **parametreleri**, **dönüş değerleri** ve **içeride izlenmesi gereken temel adımlar** sıralanmıştır.

Bir kütüphanede kitapların **ID**, **Ad**, **Yazar Adı**, **Basım Yılı**, **Durum** (rafta/ödünçte) bilgileri tutulacak, bunlarla ilgili ekleme, silme, ödünç verme, iade gibi işlemler yapılacaktır.

1) kitapEkle(...)

- **Amaç:** Kütüphaneye yeni bir kitap eklemek.
- **Parametreler:**
 - `int yeniKitapID` – Eklenmek istenen kitap ID’si.
 - `char yeniKitapAdi` – Eklenmek istenen kitabın adı (string).
 - `char yeniYazarAdi[]` – Yazar adı (string).
 - `int yeniBasimYili` – Basım yılı.
 - `int kitapID[]` – Mevcut kitapların ID’lerini tuttuğunuz dizi.
 - `char kitapAdlari[]`
 - `char yazarAdlari[][]`
 - `int basimYillari[]`
 - `int durum[]`
 - `int mevcutKitapSayisi` – Şu anda kayıtlı kitap sayısını tutan değişken.
- **Dönüş Değeri:** `void` (isteğe bağlı `int` veya `bool` ile “başarılı / hata” bilgisi döndürülebilir).
- **İzlenecek Adımlar:**
 1. Maksimum kapasite dolu mu (ör. 2000 kitap) kontrol et, doluyorsa ekleme yapma.
 2. Aynı `yeniKitapID` zaten var mı kontrol et, varsa eklemeyi iptal et.
 3. Dizide silinmiş bir indeks (`durum[i] == -1`) varsa orayı doldur; yoksa `mevcutKitapSayisi` konumuna ekleme yap.
 4. Bilgileri diziye kaydet (`kitapID[i] = yeniKitapID, vb.`).
 5. `durum[i] = 0` (rafta) varsay.
 6. Gerekirse `mevcutKitapSayisi` artır.
 7. Kullanıcıya “Kitap eklendi” benzeri bir bildirim yap.

2) kitapSil(...)

- **Amaç:** Belirli bir ID'ye sahip kitabı kütüphaneden silmek (dizide işaretlemek).
- **Parametreler:**
 - `int silinecekKitapID`
 - Diziler: `kitapID[]`, `kitapAdlari[]`, `yazarAdlari[][]`, `basimYillari[]`, `durum[]`.
 - `int mevcutKitapSayisi` – Gerekirse kitap sayacını azaltmak veya aynı bırakmak için.
- **Dönüş Değeri:** `void`.
- **İzlenecek Adımlar:**
 1. Dizide `silinecekKitapID` aranır.
 2. Bulunamazsa “Kitap yok!” mesajı ver ve fonksiyondan çık.
 3. Bulunursa, dizideki ilgili alanları `-1` veya `""` (boş string) vb. ile işaretle.
 4. `durum[i] = -1` yap (silinmiş).
 5. `mevcutKitapSayisi` bir azalt.
 6. Başarılı silindi uyarısı ver.

3) kitapOduncAl(...)

- **Amaç:** Kitabın durum bilgisini raftan (0) ödünçte (1) olarak değiştirmek.
- **Parametreler:**
 - `int oduncKitapID` – Ödünç verilecek kitabın ID'si.
 - Diziler: `kitapID[]`, `durum[]`.
- **Dönüş Değeri:** `void`.
- **İzlenecek Adımlar:**
 1. `kitapID[]`'de `oduncKitapID` aranır.
 2. Bulunamıyorsa hata mesajı ver.
 3. Bulunuyorsa, `durum[i]` kontrol edilir:
 - `durum[i] == 1` ise kitap zaten ödünçtedir, uyarı ver.
 - `durum[i] == 0` ise `durum[i] = 1` yap ve “Ödünç alma başarılı” mesajı ver.

4) kitapIadeEt(...)

- **Amaç:** Kitabı rafta (0) duruma geri almak.
- **Parametreler:**
 - int iadeKitapID
 - Diziler: kitapID[], durum[].
- **Dönüş Değeri:** void.
- **İzlenecek Adımlar:**
 1. Dizide iadeKitapID aranır.
 2. Kitap yoksa veya durum[i] == -1 (silinmiş) ise, uygun uyarı ver.
 3. durum[i] == 0 ise, zaten rafta olduğunu bildir.
 4. durum[i] == 1 ise, durum[i] = 0 yap ve “İade alındı” mesajı ver.

5) listele(...)

- **Amaç:** Tüm kitapları veya belirli bir filtreye göre (rafta/ödünçte) ekranda listelemek.
- **Parametreler:**
 - Diziler: kitapID[], kitapAdlari[], yazarAdlari[][], basimYillari[], durum[].
 - int mevcutKitapSayisi.
 - int filtre (0 = rafta olanlar, 1 = ödünçte olanlar, 2 = hepsi).
- **Dönüş Değeri:** void.
- **İzlenecek Adımlar:**
 1. Ekrana tablo başlığı (Kitap ID | Kitap Adı | Yazar | Yıl | Durum) yaz.
 2. for döngüsüyle diziyi tara:
 - durum[i] == -1 ise (silinmiş kayıt) listeleme.
 - filtre değişkenine göre durum[i] 0,1 veya her ikisine de izin ver.
 - Eşleşiyorsa satır şeklinde bastır.

6) kitapAra(...)

- **Amaç:** Kitapları ID, kitap adı veya yazar adına göre aramak.
- **Parametreler:**
 - int aramaTuru (0=ID, 1=Kitap Adı, 2=Yazar Adı).
 - char aramaMetni[] – Eğer aramaTuru 1 veya 2 ise kullanılır.

- `int aramaID` – Eğer `aramaTuru` 0 ise kullanılır.
- Diziler: `kitapID[]`, `kitapAdlari[]`, `yazarAdlari[][]`
- **Dönüş Değeri:** `void`.
- **İzlenecek Adımlar:**
 1. `aramaTuru == 0` ise `aramaID` ile `kitapID[i]` eşleşmesi aranır.
 2. `aramaTuru == 1` ise `aramaMetni` kitap adlarında aranır.
 3. `aramaTuru == 2` ise `aramaMetni` yazar adlarında aranır.
 4. Eşleşen kayıtları liste formatında bastır.
 5. Hiç kayıt bulunmazsa “Kriterle eşleşen kitap bulunamadı” mesajı.

7) kitapGuncelle(...)

- **Amaç:** Mevcut bir kitabın `kitapAdi`, `yazarAdi` veya `basimYili` gibi bilgilerini güncellemek.
- **Parametreler:**
 - `int guncellenecekKitapID`.
 - `char yeniKitapAdi`, `char yeniYazarAdi[]`, `int yeniBasimYili`.
 - Diziler: `kitapID[]`, `kitapAdlari[]`, `yazarAdlari[][]`, `basimYillari[]`, `durum[]`.
- **Dönüş Değeri:** `void`.
- **Adımlar:**
 1. Dizide `guncellenecekKitapID` aranır.
 2. Bulunamazsa “Kitap yok!” mesajı.
 3. Varsa, `durum[i] != -1` (silinmemiş) kontrolü yap.
 4. Yeni değerleri ilgili dizilere kaydet.
 5. “Güncelleme başarılı” mesajı.

8) kitapFiltreleListele(...)

- **Amaç:** `listele(...)` fonksiyonunu çeşitlendirerek `durum`, `basım yılı` veya `yazar adına` göre filtreleme yapmak.
- **Parametreler:**
 - `int filtreTipi` (0=Durum, 1=Basım Yılı, 2=Yazar Adı).
 - `int filtreDeger` (Durum ya da Basım Yılı için).
 - `char filtreMetin[]` (Yazar adı için).
 - Diziler: `kitapID[]`, `durum[]`, `yazarAdlari[][]`, vb.

- **Dönüş Değeri:** void.
- **Adımlar:**
 1. `filtreTipi == 0` ise, `durum[i]` `filtreDeger` (0 veya 1) olan kitapları bas.
 2. `filtreTipi == 1` ise, `basimYillari[i] == filtreDeger` olan kitapları bas.
 3. `filtreTipi == 2` ise, `yazarAdlari[i]` içinde `filtreMetin` geçen kitapları listele.
 4. Hiç sonuç yoksa “Filtreye uygun kitap yok” mesajı.